Python Engineering - CSV Round-Trip Testing Guide

Qué valida test_csv_round_trip(), por qué cambiar reference no provoca FAIL y cómo romperlo correctamente

1. ¿Qué es un round trip?

Un **round trip** comprueba que unos datos pueden salir del programa, pasar por un formato externo y volver a entrar sin cambiar su estructura ni sus valores.

```
DATOS EN RAM
    |
    v
save_trajectory_csv()
    |
    v
CSV EN DISCO
    |
    v
load_trajectory_csv()
    |
    v
DATOS RECUPERADOS EN RAM
```

La pregunta del test es: **¿lo que recupero es equivalente a lo que guardé?**

2. Qué NO comprueba test_csv_round_trip()

Este test no verifica que la trayectoria sea físicamente correcta. No sabe si una altura debería ser 100, 999 o 5000. Solo comprueba que el mecanismo de guardado y carga preserva los datos.

```
round-trip test checks:
original data ~= loaded data

it does NOT check:
original data == physically correct trajectory
```

La corrección física corresponde a otros tests, como `test_euler_step()`.

3. La lógica del test

```
reference = np.array([
    [100.0, 20.0, 0.0],
    [102.0, 19.019, 0.1]
])

save_trajectory_csv(test_file, reference)
loaded = load_trajectory_csv(test_file)

assert reference.shape == loaded.shape
assert np.allclose(reference, loaded)
```

Hay dos comprobaciones distintas:

Comprobación	Pregunta
shape	¿Se conserva la estructura de filas y columnas?
np.allclose	¿Se conservan numéricamente los valores?

4. ¿Por qué cambiar 100 por 999 no produce FAIL?

Porque `reference` cumple dos papeles a la vez: es el dato que guardamos y también el dato contra el que comparamos después.

```
reference = np.array([
    [999.0, 20.0, 0.0],
    [102.0, 19.019, 0.1]
])
```

El programa guarda 999, luego lo vuelve a leer, y finalmente pregunta si el 999 recuperado coincide con el 999 original.

```
input reference      -> 999
CSV stored value     -> 999
loaded value         -> 999
comparison           -> 999 ~= 999
result               -> PASS
```

El test está haciendo exactamente lo correcto: conservar el valor que recibió.

5. Reference no es lo mismo que expected físico

Concepto	Significado
reference en round trip	Dato de entrada que debe sobrevivir a guardar + cargar
expected en test_euler_step	Resultado físico/numerico independiente que esperamos obtener

En un test físico, cambiar expected puede provocar FAIL porque el cálculo se compara contra una referencia externa al resultado. En un round trip, cambiar reference cambia simultáneamente el input y la expectativa de recuperación.

6. Ejemplo de test físico

```
state = VerticalState(h=100.0, v=20.0)
state.euler_step(0.1)

result = state.state_vector()
expected = np.array([102.0, 19.019, 0.1])

assert np.allclose(result, expected)
```

Aquí `expected` viene del modelo matemático. Si se cambia a 999 sin modificar el resultado real, el test falla.

7. Para qué sirve realmente test_csv_round_trip()

Su objetivo es detectar errores de serialización y deserialización, como los que pueden aparecer al escribir columnas en orden incorrecto, perder datos, cambiar formato o leer el CSV de manera inconsistente.

Ejemplos de fallos que debería detectar:

```
- swap v and t while saving
- omit one column
- duplicate a column
- read columns in wrong order
- modify values during parsing
- change the number of rows
```

8. Ejemplo: intercambio de columnas

Supongamos que por error el writer guarda h, t, v en lugar de h, v, t.

```
# wrong writer idea
file.write(
    f'{h:.6f},{t:.6f},{v:.6f}\n'
)
```

Original:

```
[100.0, 20.0, 0.0]
[102.0, 19.019, 0.1]
```

Archivo mal escrito:

```
100.000000,0.000000,20.000000
102.000000,0.100000,19.019000
```

Al volver a cargarlo, el array ya no coincide con la referencia.

```
np.allclose(reference, loaded)
-> False
-> AssertionError
-> FAIL
```

Ese es el tipo de regresión que un round-trip test está diseñado para detectar.

9. Cómo provocar un FAIL deliberado

Para comprobar que el test realmente detecta diferencias, hay que introducir una diferencia entre el input original y el resultado recuperado.

```
loaded = load_trajectory_csv(test_file)

# deliberate corruption for learning
loaded[0, 0] = 999.0

assert np.allclose(reference, loaded)
```

Si reference[0,0] sigue siendo 100, la comparación falla y el runner mostrará FAIL.

10. Otra forma: romper temporalmente el writer

Una demostración más realista consiste en introducir temporalmente un error en `save_trajectory_csv()`, por ejemplo intercambiando columnas.

```
correct:
h, v, t

temporary bug:
h, t, v
```

Después se ejecutan los tests. El esperado es que `test_csv_round_trip()` falle. Luego se restaura el writer correcto y el test debe volver a PASS.

11. Por qué esta prueba deliberada aporta valor

Un test que siempre da PASS podría estar mal diseñado. Romper deliberadamente el sistema permite comprobar que el test tiene sensibilidad ante el tipo de fallo que pretende detectar.

```
working code -> PASS
deliberate bug -> FAIL
restore code -> PASS
```

Este ciclo valida tanto el código como la utilidad del test.

12. Qué pregunta responde cada test del EX24

Test	Pregunta
test_initial_state()	¿VerticalState se inicializa correctamente?
test_euler_step()	¿El paso de Euler produce el resultado esperado?
test_csv_round_trip()	¿Lo que guardo es lo mismo que recupero?
test_missing_file_raises()	¿Un archivo inexistente genera la excepción correcta?

13. Un test = un comportamiento concreto

Cada test debe responder a una pregunta clara. Si falla, queremos saber rápidamente qué tipo de comportamiento se ha roto.

```
physics test      -> numerical behavior
round-trip test   -> persistence integrity
exception test    -> failure behavior
```

14. Round trip y serialización

Guardar un array en CSV implica transformar datos numéricos en texto. Cargarlo implica hacer la transformación inversa.

```
NumPy float
   |
serialization
   v
CSV text
   |
parsing / deserialization
   v
NumPy float
```

El round trip verifica que esas dos transformaciones sean compatibles entre sí.

15. Limitación importante de un round-trip test

Si writer y reader cometen exactamente el mismo error de forma simétrica, un round trip podría pasar. Por ejemplo, si ambos intercambian las mismas columnas, los datos podrían volver a su forma original.

Por eso los proyectos reales combinan distintos tipos de tests: round trip, valores conocidos, validación de formato y casos independientes.

16. Cuándo usar valores conocidos adicionales

Si se quiere verificar no solo que writer y reader sean compatibles, sino que el archivo tenga exactamente el formato esperado, se puede añadir un test específico del contenido escrito.

```
save_trajectory_csv(...)
read raw text
compare against expected CSV lines
```

Ese sería otro comportamiento y, por tanto, idealmente otro test.

17. Resumen conceptual

```
test_euler_step()
-> compare calculation with known physical result

test_csv_round_trip()
-> compare loaded data with original input

changing reference in round trip
-> changes both what is saved and what is expected back
-> therefore it can still PASS

to force FAIL
-> create a mismatch between original and loaded data
```

18. Chuleta final

```
ROUND TRIP
original
 -> save
 -> file
 -> load
 -> recovered

CHECK 1
original.shape == recovered.shape

CHECK 2
np.allclose(original, recovered)

PASS
-> structure and values preserved

FAIL
-> something changed during save/load
```

Idea central: un round-trip test no pregunta si los datos son físicamente correctos; pregunta si el sistema de persistencia los conserva correctamente.